Task 1: Guided Activity, Create and Break a Retain Cycle

```swift
import Foundation
class Person {
    let name: String
    var apartment: Apartment?
    init(name: String) {
        self.name = name
    }
    deinit {
        print("Person \(name) deallocated")
    }
}
class Apartment {
    let unit: String
    var tenant: Person?
    init(unit: String) {
        self.unit = unit
    }
    deinit {
        print("Apartment \(unit) deallocated")
    }
}
var person: Person? = Person(name: "Anu")
var apartment: Apartment? = Apartment(unit: "101")
person!.apartment = apartment
apartment!.tenant = person
person = nil
apartment = nil
print("Both variables set to nil")
```

Output

```
Both variables set to nil
```

```swift
import Foundation
class Person {
    let name: String
    var apartment: Apartment?
    init(name: String) {
        self.name = name
    }
    deinit {
        print("Person \(name) deallocated")
    }
}
class Apartment {
    let unit: String
    weak var tenant: Person?
    init(unit: String) {
        self.unit = unit
    }
    deinit {
        print("Apartment \(unit) deallocated")
    }
}
var person: Person? = Person(name: "Anu")
var apartment: Apartment? = Apartment(unit: "101")
person!.apartment = apartment
apartment!.tenant = person
person = nil
apartment = nil
print("Both variables set to nil")
```

Output

```
Person Anu deallocated
Apartment 101 deallocated
Both variables set to nil
```

Explanation:

Changing tenant to weak breaks the strong reference cycle, allowing both objects' reference counts to reach zero so their deinit methods can run.

Task 2: Practice Task, Closure capture cycle

```swift
import Foundation
class Downloader {
    let name: String
    var onFinish: (() -> Void)?
    init(name: String) {
        self.name = name
    }
    func start() {
        onFinish = {
            print("Finished \(self.name)")
        }
    }
    deinit {
        print("Downloader \(name) deallocated")
    }
}
var downloader: Downloader? = Downloader(name: "File1")
downloader?.start()
downloader = nil
print("Downloader variable set to nil")
```

Output

```
Downloader variable set to nil
```

```swift
import Foundation
class Downloader {
    let name: String
    var onFinish: (() -> Void)?
    init(name: String) {
        self.name = name
    }
    func start() {
        onFinish = { [weak self] in
            guard let self = self else { return }
            print("Finished \(self.name)")
        }
    }
    deinit {
        print("Downloader \(name) deallocated")
    }
}
var downloader: Downloader? = Downloader(name: "File1")
downloader?.start()
downloader?.onFinish?()
downloader = nil
print("Downloader variable set to nil")
```

Output

```
Finished File1
Downloader File1 deallocated
Downloader variable set to nil
```

Task 3: Predict Task, When does the object die?

```swift
class Box {
    deinit {
        print("Box gone")
    }
}
var a: Box? = Box()
var b = a
var c = a
a = nil
b = nil
print("before c")
c = nil
print("end")
```

Output

```
before c
Box gone
end
```